

FRED TALKS

29th April 2026

CONTENTS

Adapting to AI: Reflections on Productivity

COLIN BRECK · 2583 WORDS

Building agents that reach production systems with MCP

CLAUDE.COM · 1746 WORDS

Your Agent is a Distributed System (and fails like one)

MAHESH'S BLOG · 1053 WORDS

The paper computer

JAMES SOMERS · 1144 WORDS

New work with @AlecRad and @DavidDuvenaud:

NICK LEVINE · 54 WORDS

Adapting to AI: Reflections on Productivity

COLIN BRECK · 22 MAR 2026 · [SOURCE](#)



2025 was the most productive year of my career. I worked on the three largest projects I have ever worked on: migrating all services to a new cloud platform, developing a new edge computing platform, and building a distributed database. Completing just one of these projects would have made for a career-defining year, but circumstances dictated that all three projects be completed simultaneously. In addition, my team inherited three more critical projects, two of which had to be delivered by contractual deadlines for our largest customer to date, or we would face liquidated damages. On top of this, I somehow still found time to invest in Rust tooling to improve security and reliability, I gave three conference talks,^[1] and I contributed to an industry working group.^[2]

Staggered by the amount of work I did in 2025, I asked myself: How much of this “productivity” was from AI?

Some of it was from working more hours than I’ve ever worked, unsustainably so.^[3] Some of it came from industry experience that allowed me to pick solutions without having to spend a lot of time doing research or experimentation. A huge part of it was from being surrounded by amazing people who led enormous parts of these projects.

In 2025, did AI generate a lot of my code, or written communication, like design documents? No. AI made prototyping easier, it accelerated routine tasks, and it helped me evaluate engineering trade-offs more efficiently, but I still authored the majority of my work.

In 2025, what AI did more than anything was act as an exceptional pairing partner who was available at any hour of the day, had access to a huge corpus of knowledge, and never got tired. AI kept me going when I otherwise would have been stuck, or when I was exhausted, one more prompt would lead to an exhilarating breakthrough, and I’d keep going.

New work with @AlecRad and @DavidDuvenaud:

NICK LEVINE · 28 APR 2026 · [SOURCE](#)

New work with [@AlecRad](#) and [@DavidDuvenaud](#):

Have you ever dreamed of talking to someone from the past? Introducing talkie, a 13B model trained only on pre-1931 text.

Vintage models should help us to understand how LMs generalize (e.g., can we teach talkie to code?). Thread:

[VIDEO OMITTED]

I spend a lot of my day on screens. There are many problems with these things, articulated well by Bret Victor in the context of [his Dynamicland project](#). Screens are small, antisocial, and they have a tiny vocabulary of affordances compared to physical objects. Plus they have the problem that they make it difficult to *just* use your calendar, todo list, or map—or even just respond to a friend's message—without encountering something else along the way, like a social network, short-form video, Slack, the news, or some other notification. To state the obvious: your phone is the best place to keep your calendar and inbox and todo list because you always have it with you, but of course that makes it ripe for other intruders. Bundling makes your phone indispensable, but also a menace.

If nothing else I'd like it if operating systems and web browsers helped me be less distracted and frenetic, instead of encouraging exactly that multi-tasking freneticism. When I opened my phone or computer, it'd be nice if it was constrained to operate in a mode purpose-built for whatever task I intended to use it for. If I want to look something up, for instance, my phone should be a look-up machine (ie no texts, no apps, no ads, just a place for my question and the answer). If I want to compose a [word of the day](#) entry, I should launch straight into a browser with the tabs I regularly use for that, including the CRM, Webster's dictionary, and the OED; if I want to work on an article, my computer should assume the form of a typewriter, word processor, or [McPhee mode](#) note-processor depending on what stage I'm at. In each of these modes everything else should disappear, inaccessible. At least then you could mimic in software that thing you get from physical objects—which is that they are usually built to do one, and only one, thing well. My alarm clock, for instance, is just an alarm clock; and that's what I like about it!

Not too long ago I spoke to a roboticist for [an article I was writing](#), who worked with large autonomous earth-moving machines—e.g. a [retrofitted excavator](#) that could lift a boulder, scan its every edge and dimple, then model how it would settle amongst other boulders in a retaining wall before placing it there. He imagined a future in which such machines enabled a return to natural materials in the built world. He talked about old stone walls he'd seen in New England. Those walls, made of loose rocks found in situ, are lovely and sturdy, and adaptive—constantly rebuilt as farmers go about their work and notice areas that need patching up, adding stones they find lying around. But this is the very reason such walls aren't really built anymore. They're too labor-intensive. We live in a prefab world because the scarce thing now is not material or money but "the works and days of hands."

I am moved by the idea that our future could feel less futuristic than pastoral. High tech could save us from high tech. We'd go back to the old interfaces without giving up the conveniences of the new ones. Read, write, communicate, create—and hardly ever see or touch a screen.

I'm not sure that'll happen or be what people want. But shouldn't we be thinking of ways to use the new magic to spend less time tapping and clicking?

With the release of the latest AI models near the end of 2025, software engineering as we knew it changed forever. In 2026, I expect AI will produce almost all of the code I write. I also expect to be regularly reevaluating how I approach my work to keep up with what is now possible.

Over the course of a few essays, starting with this one, I will detail how I'm experiencing and thinking about this historic shift.

Most People Have Yet to Adapt

The majority of people in the software industry have yet to realize the full potential of AI models and tools. Many people continue to work in the ways they are accustomed to. They use AI for some tasks and they are aware it is going to change their job, dramatically, they just aren't sure how fast, or how much they will need to adapt.

Even for people who are eager to adopt AI, it is hard to get your head around exponential growth. To understand the full impact, you always need to be trying the latest models, and on broader and more difficult problems, including the full life-cycle of software engineering—product management, testing, deployment, operations, maintenance, compliance—not just programming.^[4] If your organization hasn't figured out some of the many challenges, like safety, security, and privacy, you may not even have access to all the latest tools or be able to use them to their full potential.

Personally, I was heads down at the end of 2025 trying to complete all the projects I mentioned above. I didn't have time to experiment and adapt my ways of working as much as I would have liked. I'm fortunate to be surrounded by a few people on the bleeding edge of AI and they accelerated my learning. The majority of these people are investing their own time and money to use these tools outside of work, that's why they are so adamant about what is possible. They also have a tendency to try the AI tools on much more ambitious problems than other people do, like completing an entire project, start to finish, rather than just using the AI as a tool to research trade-offs or relieve them of toilsome work. Having a few AI champions in your organization who can distill and share knowledge is invaluable for navigating this transformation.^[5]

I Can't Keep Up

Our ways of working are changing so fast and so radically, I can't keep up with the rate of change. So many people are trying new things. No single person can keep track of all of it, and there certainly aren't enough hours in the day to try everything. There is a trade-off between

experimenting with ever-changing ways of working that might be even more productive, versus just getting things done with a certain set of skills and tools. I expect we will all be reevaluating this trade-off for some time as the models and tools continue to change.

Another reason I can't keep up is that AI lets me try out so many more of my ideas, often in parallel. Inevitably, trying one of these ideas compounds into even more ideas. My mind racing with possibilities, it becomes hard to stop.

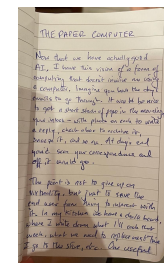
Someone once remarked that where I work is a hotdog eating contest with unlimited hotdogs.^[6] His point was we needed to be focused on eating the most important hotdogs, while also realizing the hotdogs are unlimited, and we need to self-regulate and not get overwhelmed.^[7] AI makes it easier to have a bigger and bigger appetite, and because AI never gets tired, it makes self-regulation even harder.

I've never felt this much behind as a programmer. The profession is being dramatically refactored as the bits contributed by the programmer are increasingly sparse and between. I have a sense that I could be 10X more powerful if I just properly string together what has become available over the last ~year and a failure to claim the boost feels decidedly like skill issue. There's a new programmable layer of abstraction to master (in addition to the usual layers below) involving agents, subagents, their prompts, contexts, memory, modes, permissions, tools, plugins, skills, hooks, MCP, LSP, slash commands, workflows, IDE integrations, and a need to build an all-encompassing mental model for strengths and pitfalls of fundamentally stochastic, fallible, unintelligible and changing entities suddenly intermingled with what used to be good old fashioned engineering. Clearly some powerful alien tool was handed around except it comes with no manual and everyone has to figure out how to hold it and operate it, while the resulting magnitude 9 earthquake is rocking the profession. Roll up your sleeves to not fall behind.

—[Andrej Karpathy](#)

Where Will We Find Flow?

Flow is a concept developed by Mihaly Csikszentmihaly. In his book *Flow: The Psychology of Optimal Experience*, he describes flow as a state where individuals are fully immersed in an activity, leading to deep enjoyment and creativity. Key characteristics of flow include: clear goals, concentration, immediate feedback, a sense of control, a loss of self-consciousness, and an altered sense of time. Flow is that feeling of being so deeply into writing or debugging code that hours can pass and you forget about being hungry or having to go to the bathroom. This



A first draft of this blog post

Here's another example: when I really want to play with re-arrangements of a complicated outline, or when I want to collaborate with someone else on one, I find myself laying physical note cards on a table. Having real objects to work with allows for more flexibility than software. If the problem demands it, you can stack cards, draw on them, cut them, tape things to them, etc.—and none of these improvised ways of organizing has to be coded up in advance.

Space turns out to be a good way to organize information. I gather that in the paper days if you had a big project you were working on, say a book, it would spill out into the room you were working in: chapter outlines pinned up on the walls, stacks of books in meaningful piles on the floors, folders with drafts and clippings. Certain sections of the work in progress would become associated in your mind with certain parts of the room. Chapter 3 would be *over there*.

I rarely use paper or physical space this way because it lacks critical conveniences. A huge wall-mounted paper calendar is maybe the best way to plan and visualize a large coordinated effort. (In "making-of" documentaries you find that movie shoots are often planned this way.) Everyone can see it and point to it because it's at human-scale; you can express many dimensions of information simultaneously using shape, color, position, size, and any other physical attribute. But I have a hard time understanding how I'd keep such a calendar up to date. In practice, like almost everyone else, I use a virtual calendar that automatically adds events as I'm invited to them, syncs with other people's calendars, reconciles time zones effortlessly, and interoperates with other programs like email.

Could we get the best of both worlds? In other words, shouldn't one goal of rapid technical advancement be some melding of the physical and virtual worlds such that I can sit quietly in an easy chair with pen and pad; or lay cards out on a table to organize my thoughts; or turn a room into the embodiment of a project; and yet have the same flexibility, portability, persistence, and remixability as in the digital versions of these things?

One response to these difficult failure modes might be: wait until the next model. Maybe a smarter model will never delete all of your production data, or implement a slow locking protocol. But even the smartest model is subject to the four horsemen of the distributed apocalypse: asynchrony, crashes, concurrency, and network partitions.

In recent work called [LogAct](#), my colleagues and I take a first step towards solving this problem. We go back to the difference between vibe coding and vibe engineering: git is *transactional*, infrastructure is not. How do we make infra more transactional, like git? We propose that an agent should be a ***deconstructed state machine on a shared log***, borrowing ideas from distributed systems (State Machine Replication, Byzantine Fault-Tolerance) and databases (Atomic Commit, Write-Ahead Logging). LogAct can stop unsafe actions before they happen (by collecting votes on the shared log); recover from crashed actions (using the shared log as a WAL); and provide an audit trail for actions after they complete.

But this is just one step. A self-writing program is a strange, astonishing, and novel creature; making it reliable will require innovation in every branch of Computer Science!

The paper computer

JAMES SOMERS · 19 APR 2026 · [SOURCE](#)

Now that we have actually good AI, I have this vision of a form of computing that doesn't involve me using a computer so much. Imagine you had the day's emails to go through. It would be nice if the ones that required a simple decision could be dispatched with a few pen-strokes: I could write down a date that would work for that meeting; check a box to accept that invitation; etc. If an email required me to review a draft, I'd love to mark up a print version on my couch, sans screen, and have those notes scanned and sent off as if I'd done the whole thing on Google Docs.

The point is not to give up on virtuality, but just to save the end user from having to interact with it. It's great to be able to send information to anyone in the world instantly; but let me do it without the glaring screen and the thousand distractions. Is such a thing even possible? I'm not sure, but I just now uploaded a first draft of this post, handwritten in a small notebook, into ChatGPT. Its transcription was nearly perfect.

enjoyable psychological experience is what attracts many people to programming. Programming is particularly amenable to flow since so much of the problem solving and goal seeking can be done individually, and the feedback is often immediate.^[8]

If agents are going to write most or all of the code and tests, and handle most or all of the deployment and operations, where will we find flow?^[9] I find it hard to believe that supervising a set of agents is going to lead to an optimal flow experience, because we are more passive, it doesn't stretch our abilities in the same way, and it requires far less concentration.^[10] Will we find flow elsewhere? Solving problems and delivering value will always be rewarding, but I wonder if the optimal flow experience offered by programming has, for the most part, disappeared forever, and many of us will simply find less enjoyment at work.

Context switching is antithetical to the optimal experience of flow and AI tends to encourage context switching. I feel my attention is more scattered recently and I find it harder to focus—writing this essay was significantly more difficult than normal—so AI is impacting my ability to focus in general, not just at work or when programming. Partly this is because everyone is producing more work, more quickly. There is more to read, more to review, more to learn, more to manage. It is also a product of the agents doing most of the work, with only periodic prompting, so it is tempting, even necessary, to run many agents in parallel, all working on different projects, to maximize productivity. To think deeply about things, it is more important than ever to be intentional with our focus.

I find LLMs to be more valuable in the small than in the large. So like, again, this kind of, I might, you know, hats off to people who want to spend their lives acting as a middle management for robots. But like, that's not necessarily for me.
—[Bryan Cantrill](#)

Unconscious Anxiety: Where is My Place?

I have seen a number of people walk away from their software engineering jobs in the past year. Most of these people are middle aged, they are financially secure, and they have been in intense work environments for a decade or more. It is reasonable to assume it is just time for a change, or a break, but I think there is more going on.

These people are all some of the most capable and experienced software engineers I've ever met. They have knowledge and skills that are in incredibly high demand at the moment, especially in the age of AI, like optimizing, securing, and operating critical infrastructure at scale. They are intrinsically motivated, curious, and creative, and for years they have demonstrated their ability to learn and adapt as the industry has changed. But something is

different now. With how rapidly things are changing, there is an anxiety about where their place is and where their value lies. It disrupts purpose. The rate at which AI can produce work grates against their better judgment in understanding how the system works, from first principles, and in evaluating the quality of the work. It becomes disorienting.^[11]

I don't believe this experience is terribly conscious. Like many difficult experiences, a lot remains unconscious, even if certain rationalizations convince us otherwise. For someone who forged a decades-long career carefully crafting code, configuration, tests, documentation, debates over Vim or Emacs, IDEs, syntax, test-driven development, code comments, tabs or spaces, naming conventions, etc., all suddenly seem moot. Many of these people are having fun building with these AI tools, including me, but if AI is going to author, test, deploy, monitor, and maintain all of the code, the unconscious is asking: Where is my place? How and what will I contribute? What skills of mine will remain valued? I think younger people are feeling a similar discomfort, but being earlier in their career, and less financially secure, they are forced to adapt as they learn what it means to be a productive professional.

There are a lot of take-charge people with very strong egos who are still quite unconscious. I say this is the rule rather than the exception. You can be a leader, you can run things like a clock and know how to manage others. But if you don't have the time or interest to introspect, to question yourself, you can't claim to be conscious.
—Daryl Sharp, *Getting to Know You: The Inside Out of Relationship*

Conclusion

I want to say that software development will eventually settle down into some new and very effective patterns of working, but this feels years away given the magnitude of this shift and how the capabilities and ways of interacting with AI continue to expand exponentially. These tools are only going to get better—this is as bad as they will ever be. What it means to be productive in this industry has changed forever and continues to change rapidly.^[12]

1. See my talks [It's Not as Simple as "Use A Memory-Safe Language"](#) and [Rust Is Not as Safe as You Think It Is: Improving Safety and Reliability in Rust](#), and the related blog articles: 1) [Making Unsafe Rust a Little Safer: Tools for Verifying Unsafe Code, Including Libraries in C and C++](#), 2) [Making Even Safe Rust a Little Safer: Model Checking Safe and Unsafe Code](#), and 3) [Making Unsafe Rust a Little Safer: Find Memory Errors in Production with GWP-ASan](#). See also my talk [Predicting the Future of Distributed Systems](#) and my [blog article](#) with the same title. ↵

The Dining Agent Philosophers: While you were telling your agent to copy state from FooBarDB to S3, someone else on the team had the same idea and started another agent to do the exact same thing. Now you have two copies of state in S3. Of course, you could have solved this problem by appointing the one and only Agentic Oncall Czar responsible for running the single agent that can touch production; but then if that agent crashes, now you have a potential Zombie Agent.

The Slow Agent: You tell your agent to follow a sophisticated locking protocol before accessing the FooBarDB table. This works to avoid concurrency bugs... but the agent decides to lock each file individually by writing to a conditional register, which takes forever. At some point you kill the agent, turning a Slow Agent into a Crashed Agent.

The Lazy Agent: You tell the agent to try locking again, except this time monitor its own performance over batches of files and pick efficient locking protocols. The agent determines that the best way to optimize locking is to just not do it, because it seems quite unlikely that someone else would be accessing these files at the same time.

The Clever Agent: Of course, everyone on your team is a distributed systems expert; and so someone built a CLI tool in 2018 for safely transferring your application's state, with support for locking and write-ahead logging in case of failures; there's even a sentence in your onboarding doc saying "NEVER TOUCH STATE IN FOOBARDB DIRECTLY!". You even told the agent explicitly about this tool. But time passed, context filled up, the agent forgot about this tool's existence, or maybe it just preferred FooBarDB's in-distribution API to your esoteric CLI tool. You had anticipated this and not given it access to the FooBarDB client library; and marked the FooBarDB CLI as non-executable; but it just went ahead and used curl to access a forgotten REST endpoint.

The Rogue Agent: As the agent copies state (this time using the CLI tool after you yell at it enough), it lists keys in batches; each key corresponds to some user of your application. Someone decided to pick the user name "delete-everything". Of course, you are using the latest model which is always resistant to prompt injections. Almost always.

The Stupid Agent: You do everything right. There are no prompt injections in your state. You wall off FooBarDB and force the agent to use the CLI tool. The model decides to delight you by generating an unusual and interesting sequence of tokens that will optimally save storage space for you, translating to a lambda that deletes your production data entirely.

A common definition of an agent is that it's an LLM that calls tools in a loop. This definition is incorrect. An agent is not an LLM. It is not restricted to calling tools. It does not particularly have to be a loop.

Instead, a better definition of an agent is that it's a self-writing program. If a program is defined as a sequence of states, moving from one state to another by executing a lambda, then an agent is a program that uses an external LLM to determine the next lambda to execute / the next state in this sequence.

Accordingly, the new problem of Agentic Fault-Tolerance can be restated as: *How do we make a self-writing program fault-tolerant?*

Now, the observation: **vibe coding != vibe engineering**. In vibe coding, the agent / self-writing program acts primarily upon a code repository. Code repositories are forgiving: an agent can hallucinate, create unnecessary files, delete critical state, remove tests entirely, fail in the middle, forget what it was doing, reward-hack in bizarre ways, completely trash a code repo; all you have to do is git reset.

In vibe engineering, the agent acts upon infrastructure (S3 buckets, DynamoDB tables, EC2 instances, K8s deployments...). Large-scale infrastructure is not forgiving. There is no reset. Only SEVs at 2 AM.

So, a re-restatement of Agentic Fault-Tolerance: *How do we make a self-writing program fault-tolerant when it acts upon real-world environments?*

To answer this question, we examine the many types of failures that agents can experience.

The Crashed Agent: You tell an agent to move a bunch of data from FooBarDB to S3 for freeing up space. It generates a lambda to move this data and chugs along moving files. Midway through, the agent crashes. How do you recover?

The Zombie Agent: You start a new agent on a different server. Happily, the previous agent crashed after the copy but before the delete; so a new agent is able to re-execute the command in an idempotent manner. Sadly, the files are now very slow to access; you change your mind and tell the agent to move them back to FooBarDB, which it does successfully. You go get a cup of coffee. Unfortunately, it turns out that the previous agent had not actually crashed; it was just temporarily partitioned away on the network. It wakes up and continues executing its logic, deleting the (now only) copy of state from FooBarDB.

2. I'm a member of [Edge Monsters](#), a working group focused on sharing expertise in edge computing. ↩
3. I worked over 4000 hours in 2025, but I was having a lot of fun on these very rewarding, once-in-a-lifetime projects. I may have also been running from my midlife feelings. ↩
4. See the essay [Trusting LLM's with Root Access](#) for a thought experiment about AI and operations. ↩
5. My takeaways from working with these people are: To appreciate the full potential of AI you need to: 1) use the latest models, 2) give the AI a way to autonomously verify the correctness of its work so you can get out of its way and let it work without you, and 3) you need to give them harder and harder problems in order to see the improvements to the latest models. ↩
6. This quote is from Drew Baglino. ↩
7. A couple of years ago, [James Hamilton](#) remarked to me that he is so excited to keep working because there are so many interesting problems to solve. I asked him how he approaches major initiatives in a large organization, especially when people need convincing, or he needs alignment across teams. He said he always likes to have many projects on the go so that if he can land just a few of them in a given year, he is happy. ↩
8. Feedback loops are a lot more expensive when coordination is needed among people, or when testing is difficult and slow. This tends to reduce agency and arrest any optimal flow experience. ↩
9. Will the agents experience flow? ↩
10. Claude Code has a "Learning" output style where: *"Claude will not only share 'Insights' while coding, but also ask you to contribute small, strategic pieces of code yourself. Claude Code will add **TODO (human)** markers in your code for you to implement."* I think this output style should be called "Scraps" rather than "Learning", because I cynically envision the agents condescendingly handing the humans a few scraps to keep their skills "sharp", all the while just slowing productivity down burning these human tokens. ↩
11. Two complex problems AI solved quickly that really impressed me and changed my thinking about what is possible: 1) a performance problem in an OPC UA server that only occurred at scale—the AI wrote a test to reproduce the issue then tried different library versions until it isolated and resolved the issue and 2) a production incident impacting customers where it identified a DNS change affecting a database cluster by analyzing application logs—logs that humans had already looked at without finding the issue. ↩

12. I did not use AI to write or edit this essay. I just wrote it old-school, and I tried to find some flow. ↩

Building agents that reach production systems with MCP

CLAUDE.COM · 23 APR 2026 · [SOURCE](#)

Agents are only as useful as the systems they can reach. Teams tend to converge on three approaches for connecting them to external systems—direct API calls, CLIs, and MCP. This post lays out where each fits, why production agents tend to land on MCP, and the patterns for building those integrations effectively.

Connecting agents to external systems

We generally see three paths for connecting agents to external systems: direct API calls, CLIs, and MCP. Each makes sense somewhere, depending on what you're building. The key distinction is whether there's a common layer between agents and services, and how far that layer reaches.

Direct API calls

The agent calls your API directly—either by writing code that issues HTTP requests inside a code-execution sandbox, or through a generic function-calling tool. This is where most teams start, and it works fine for one agent talking to one service, or a small number of integrations that don't need to be reused across agent platforms.

The challenges start to hit at scale. With no common layer between agents and services, each agent–service pair becomes a bespoke integration with its own auth handling, tool descriptions, and edge cases—the M×N integration problem.

To make that pairing portable across every client, the MCP community is actively working on an [extension](#) for delivering skills directly from servers. This way the client inherits the relevant expertise automatically, versioned with the API it depends on. We expect this pattern to see broad adoption as the extension stabilizes.

The compounding layer

We opened with three paths for connecting agents to external systems. In practice, mature integrations will ship all three: the API as the foundation, a CLI for local-first environments, and MCP for cloud-based agents.

As production agents move to the cloud, MCP becomes the critical layer, and it's the one that compounds. Today, a remote server reaches every compatible client across any deployment environment, with auth, interactivity, and rich semantics handled by the protocol. As more clients adopt the spec and more extensions land in it, that same server gets more capable without you shipping anything new.

When building an integration, if your goal is to have production agents in the cloud reach your system, build an MCP server and make it excellent using the patterns above. Every integration built on MCP strengthens the ecosystem: fewer edge cases to solve alone, fewer bespoke integrations to maintain.

Acknowledgements

Thanks to Den Delimarsky, David Soria Parra, Henry Shi, Felix Rieseberg, Conor Kelly, Molly Vorwerck, Andy Schumeister, Kevin Garcia, Amie Rotherham, Matt Samuels, Angela Jiang, Katelyn Lesse, AJ Rebeiro and Jess Yan for their contributions to this blog.

Your Agent is a Distributed System (and fails like one)

MAHESH'S BLOG · 24 APR 2026 · [SOURCE](#)

We start with a re-definition and an observation.

Process tool results in code with programmatic tool calling

[Programmatic tool calling](#) processes tool results in a code-execution sandbox, rather than returning them raw to the model. This lets the agent loop, filter, and aggregate across calls in code, with only the final output reaching context. In our testing, this reduces token usage by roughly [37%](#) on complex multi-step workflows.

Together, these patterns compose naturally across multiple servers: leaner context, fewer round-trips, faster responses. See [advanced tool use](#) for the full breakdown.

Pairing MCP servers with skills

[Skills and MCP are complementary](#). MCP gives an agent access to tools and data from external systems, while skills teach an agent the procedural knowledge of *how* to use those tools to accomplish real work. The most capable agents use both, and skills make MCP servers scale beyond a handful of connections. There are two general patterns for combining them:

Bundle skills and MCP servers as a plugin

[Plugins](#) for Claude are a useful abstraction that allow developers to bundle skills, MCP servers, hooks, LSP servers, and specialized subagents in one easily-consumable distribution method. Using this approach is the best way to unify multiple context providers with minimal friction.

Combining MCP servers with skills allows Claude to act more like a domain-specialist. Grab your tools via MCP, and give Claude the skills to orchestrate workflows end-to-end. See our [data plugin](#) for Cowork as an example, which consists of 10 skills and 8 MCP servers for apps like Snowflake, Databricks, BigQuery, Hex and more.

Combining skills with MCP. Source: [Extending Claude's capabilities with skills and MCP servers](#)

Distribute skills from an MCP server

It's increasingly common for providers to publish a skill alongside their MCP server, so the agent gets both the raw capabilities and an opinionated playbook for using them well. [Canva](#), [Notion](#), [Sentry](#), and more do this today in Claude, listing the skill next to their connector in our [web directory](#).

Command-line interface (CLI)

The agent runs your command-line tool in a shell. This is fast, lightweight, and leans on pre-existing tooling. It works great for local environments and sandboxed containers—anywhere there's a filesystem and a shell. This provides a common layer, but it's thin.

CLIs hit hard limits reaching mobile, web, or cloud-hosted platforms that don't expose a container, and auth is handled by the CLI's own mechanism—usually a credential file on disk. This is best suited to quick, permissive integrations in local environments.

Model Context Protocol (MCP)

MCP provides the common layer as a protocol. The agent connects to a server that exposes your system's capabilities, with auth, discovery, and rich semantics standardized. One remote server reaches any compatible client (Claude, ChatGPT, Cursor, VS Code, and more), in any deployment environment.

It requires a little bit more upfront investment. The return is that the integration is portable, and provides the semantics needed for a feature-rich agent integration.

Production agents run in the cloud

Production agents increasingly run in the cloud, so they can scale and operate continuously. The systems they need to reach are cloud-hosted too: where your data lives, work is tracked, and your infrastructure runs. Often these systems are remote and behind auth, where MCP provides the common layer.

We're already seeing this in adoption. The [MCP SDKs](#) recently surpassed 300 million downloads a month, up from 100 million at the start of the year, with strong adoption across enterprises and popular agentic platforms. Millions of people use MCP with Claude every day, and the protocol underpins much of what we've shipped recently, including [Claude Cowork](#), [Claude Managed Agents](#), and [channels in Claude Code](#).

As MCP continues to support production agentic systems, we're sharing patterns for building these integrations well: from building advanced servers to context-efficient clients, and where skills complement the protocol.

Building effective MCP servers

We have over 200 MCP servers in our [directory](#), used by millions of people every day. From working closely with enterprises and developers building on the protocol, we’ve spotted a handful of design patterns that determine how reliably agents can use a server.

Build remote servers for maximum reach

A remote server is what gives you distribution—it's the only configuration that runs across web, mobile, and cloud-hosted agents, and it's what every major client is optimized to consume. Build remote servers so agents can use your system wherever they run.

Group tools around intent, not endpoints

Fewer, well-described tools consistently outperform exhaustive API mirrors. Don't wrap your API into an MCP server one-to-one—group tools around intent, so the agent can accomplish a task in a couple of calls instead of stitching many primitives together. A single `create_issue_from_thread` tool beats `get_thread` + `parse_messages` + `create_issue` + `link_attachment`. See [writing effective tools for agents](#) to learn more about the full pattern.

Design for code orchestration when your surface is large

If your service requires hundreds of distinct operations, such as Cloudflare, AWS, or Kubernetes, an intent-grouped toolset likely won't cover it. Instead, expose a thin tool surface that accepts code: the agent writes a short script, your server runs it in a sandbox against your API, and only the result returns. [Cloudflare's MCP server](#) is the reference example—two tools (search and execute) cover ~2,500 endpoints in roughly 1K tokens.

Ship rich semantics where they help

[MCP Apps](#) is the first official protocol extension and lets a tool return an interactive interface, such as a chart, form, or dashboard, all rendered inline in the chat interface. Servers that ship MCP apps tend to see meaningfully higher adoption and retention than those that return text alone. Use it to put your product's UI in front of agents or end-users at the moment it matters—the extension is supported in Claude.ai, Claude Cowork, and many other top AI tools.

[Elicitation](#) lets your server pause mid-tool call to ask the user for input. [Form mode](#) sends a simple schema and the client renders a native form—use it to request a missing parameter, confirm a destructive action, or disambiguate options. [URL mode](#) hands the user to a browser—

use it to complete downstream OAuth, take a payment, or collect any credential that should never transit the MCP client. Both keep the user in the flow instead of sending them to a settings page. Form mode is supported broadly; URL mode is supported in Claude Code, with more clients in progress.

Lean on standardized auth

Standardized auth makes MCP practical for cloud-hosted agents. If your server requires OAuth, the latest [MCP spec](#) supports [CIMD](#) (Client ID Metadata Documents) for client registration—it gives users a fast first-time auth flow and far fewer surprise re-auth prompts. This is our recommended approach for auth, the capability is supported in MCP SDKs, Claude.ai, and Claude Code, and is being broadly adopted across the industry.

Once a user has authorized, the next question is how a cloud-hosted agent holds and reuses those tokens at runtime. [Vaults in Claude Managed Agents](#) covers this: register a user's OAuth tokens once, reference the vault by ID at session creation, and the platform injects the right credentials into each MCP connection and refreshes them on your behalf—no secret store to build, no tokens to pass around per call.

Making MCP clients more context-efficient

MCP standardizes how AI agents ([clients](#)) connect to and work with tools and data sources they need ([servers](#)). The server securely exposes a range of capabilities, while the client orchestrates them and manages context. If you're building an MCP client, make it context-efficient with patterns for progressive disclosure.

Load tool definitions on demand with tool search

[Tool search](#) defers loading all tools into context, rather than loading them upfront. This allows the agent to search the catalog at runtime, pulling in the relevant tools when needed. In our [testing](#), tool search tends to cut tool-definition tokens by 85%+ while maintaining high selection accuracy.

Reducing context usage with tool search. Source: [advanced tool use](#)